

TwinLoop 后端接口文档

面向前端设计与联调 | 代码盘点日期：2026-09-14

这份文档把当前仓库中已经写出的 FastAPI

接口按功能分类，逐一说明用途、参数和返回值。没有编程经验也可以按照“请求地址 + 参数 + 示例”理解；前端开发时优先使用本文件中标注为“已实现”的 HTTP 接口。

阅读方法：先看公共约定，再按登录、知乎资料、画像初始化、性格测评、领域选择和聊天记录的顺序阅读。所有地址都假定后端服务运行在同一域名下。

1. 公共约定

请求头

请求头	是否必须	用途
Cookie: twinloop_session	正式登录接口必须	知乎 OAuth 登录成功后由后端设置，前端无需读取内容。
X-Request-Id	写请求建议传	用于排查问题；不传时后端生成。
X-User-Id	本地演示接口使用	旧版演示接口和 runtime 查询接口用它识别当前用户。生产登录优先使用 HttpOnly Cookie。

响应格式有两种：认证、知乎授权和知乎数据接口通常返回统一包装；早期画像演示接口和 runtime 接口直接返回 JSON。

统一成功响应

```
{
  "request_id": "req_xxx",
  "data": { ... }
}
```

统一失败响应

```
{
  "request_id": "req_xxx",
  "error": {
    "code": "ERROR_CODE",
    "message": "给用户看的说明",
    "retryable": false
  }
}
```

2. 登录、授权与系统状态

GET /health

检查后端是否启动。适合前端进入页面时做连通性检查。

项目	说明
是否需要登录	不需要
请求参数	无

响应示例

```
{ "ok": true, "service": "twinloop-api", "time": "2026-09-14T...+00:00" }
```

POST /api/v1/sources/zhihu/connect

创建知乎授权地址。前端拿到 authorization_url 后跳转浏览器。

项目	说明
是否需要登录	不需要
请求参数	body.scopes: 字符串数组, 可选, 默认 [profile, answers]; body.request_id 可选。

请求示例

```
{ "scopes": [ "profile", "answers", "request_id": "req_demo" ] }
```

响应示例

```
{ "request_id": "req_demo", "data": { "authorization_url": "https://www.zhihu.com/oauth/...", "state_id": "state_xxx" } }
```

前端提示：不要把 authorization_url 改写成普通接口请求，应该让浏览器直接跳转。

GET /api/v1/sources/zhihu/callback

知乎授权回调。通常由知乎服务器跳回，不由前端主动调用。

项目	说明
是否需要登录	需要 OAuth state，不需要前端登录 Cookie
请求参数	查询参数 authorization_code 或 code（二选一）、state。

请求示例

浏览器访问 /api/v1/sources/zhihu/callback?authorization_code=...&state=...

响应示例

成功时 302 跳转到前端地址，并设置 HttpOnly 会话 Cookie；失败时返回统一 error。

前端提示：前端只需要在回跳后的页面调用 /api/v1/me。

GET /api/v1/me

读取当前登录用户、知乎连接状态和数字分身状态。

项目	说明
是否需要登录	需要登录 Cookie
请求参数	无

响应示例

```
{ "request_id": "req_xxx", "data": { "user_id": "zhihu_123", "avatar_id": null, "avatar_status": "not_created", "zhihu_connected": true, "zhihu": { "fullname": "小明", "avatar_url": "https://...", "headline": "后端工程师" } } }
```

POST /api/v1/auth/logout

退出当前平台登录。只清除本地会话，不撤销知乎授权。

项目	说明
是否需要登录	不登录也可调用，接口幂等
请求参数	body.request_id 可选。

请求示例

```
{"request_id":"req_logout"}
```

响应示例

```
{"request_id":"req_logout","data":{"logged_out":true}}
```

GET /v1/me

旧版演示接口：读取内存中的画像状态。前端新页面不建议依赖它。

项目	说明
是否需要登录	通过 X-User-Id 识别
请求参数	请求头 X-User-Id，可省略，默认 local-demo-user。

请求示例

```
X-User-Id: user_123
```

响应示例

```
{"user_id":"user_123","twin":{"status":"not_started","revision":null}}
```

前端提示：这是兼容旧原型的直接 JSON 接口，与 /api/v1/me 的正式登录响应不同。

3. 知乎资料读取

下面接口都要求已经完成知乎授权。知乎 access token 只保存在服务端会话中，前端不能也不应该把 token 放进请求。

GET /api/v1/zhihu/contents

读取当前用户的回答、文章、视频、想法或问题列表。

项目	说明
是否需要登录	需要登录和有效知乎授权
请求参数	type=all/answer/article/zvideo/pin/question; limit 1-50，默认20; offset 翻页游标; sort=ts 或 like_count; order=desc 或 asc。

请求示例

```
?type=answer&limit=20&offset=0&sort=ts&order=desc
```

响应示例

```
{"request_id":"req_xxx","data":{"items":[{"id":"...", "title":"...", "content":"..."}], "next_offset":"20"}}
```

前端提示：首次 offset 传 0，后续把响应中的 next_offset 原样带回。

GET /api/v1/zhihu/followees

读取当前用户关注的人。

项目	说明
是否需要登录	需要登录和有效知乎授权
请求参数	limit 1-50，默认20; offset 默认0。

请求示例

```
?limit=20&offset=0
```

响应示例

```
{"request_id":"req_xxx","data":{"items":[{"id":"...","name":"..."}],"next_offset":"20"}}
```

GET /api/v1/zhihu/favlists

读取当前用户收藏夹列表。

项目	说明
是否需要登录	需要登录和有效知乎授权
请求参数	limit 1-50，默认20。

请求示例

```
?limit=20
```

响应示例

```
{"request_id":"req_xxx","data":{"items":[{"url_token":"tech","title":"技术收藏","item_count":12}]}}
```

GET /api/v1/zhihu/favlists/{url_token}/contents

读取指定收藏夹中的内容。url_token 来自收藏夹列表。

项目	说明
是否需要登录	需要登录和有效知乎授权
请求参数	路径参数 url_token； limit 1-50； offset 默认0。

请求示例

```
/api/v1/zhihu/favlists/tech/contents?limit=20&offset=0
```

响应示例

```
{"request_id":"req_xxx","data":{"items":[{"id":"...","title":"..."},"next_offset":"20"}}
```

GET /api/v1/zhihu/collections

读取近期收藏。当前接口无分页。

项目	说明
是否需要登录	需要登录和有效知乎授权
请求参数	limit 1-50，默认20。

请求示例

```
?limit=20
```

响应示例

```
{"request_id":"req_xxx","data":{"items":[{"id":"...","title":"...","created_at":"..."}]}}
```

4. 领域目录与选择

GET /v1/domains

读取领域目录，也可按关键词、父级或层级筛选。

项目	说明
是否需要登录	当前通过 X-User-Id 兼容调用

项目	说明
请求参数	q 关键词; parent_id 父级 ID; level=root/category/leaf。

请求示例

?q=人工智能&level=leaf

响应示例

```
{
  "catalog_version": "domains-2026-01",
  "items": [
    {
      "id": "computer.03.01",
      "label": "人工智能与机器学习-基础概念",
      "parent_id": "computer.03",
      "level": "leaf",
      "aliases": ["人工智能与机器学习"],
      "description": ""
    }
  ]
}
```

GET /v1/domains/selections/me

读取当前用户已选择的兴趣和擅长领域。

项目	说明
是否需要登录	通过 X-User-Id
请求参数	无

请求示例

X-User-Id: user_123

响应示例

```
{
  "catalog_version": "domains-2026-01",
  "revision": 1,
  "interests": [
    {
      "domain_id": "computer.03.01",
      "interest_level": 3,
      "proficiency": null
    }
  ],
  "expertise": []
}
```

PUT /v1/domains/selections/me

整体替换兴趣和擅长领域选择。

项目	说明
是否需要登录	通过 X-User-Id
请求参数	expected_revision; interests[]; expertise[]。interest_level 为0-3; proficiency 可为 novice/familiar/working/advanced/unknown。

请求示例

```
{
  "expected_revision": 0,
  "interests": [
    {
      "domain_id": "computer.03.01",
      "interest_level": 3
    }
  ],
  "expertise": [
    {
      "domain_id": "computer.05.02",
      "interest_level": 2,
      "proficiency": "advanced",
      "notes": "熟悉分布式系统"
    }
  ]
}
```

响应示例

```
{
  "catalog_version": "domains-2026-01",
  "revision": 1,
  "interests": [...],
  "expertise": [...]
}
```

前端提示：这是整体替换而不是单项追加；revision 不一致会返回 409。

GET /v1/domains/{domain_id}

读取一个领域节点的详情。

项目	说明
是否需要登录	不需要登录
请求参数	路径参数 domain_id。

请求示例

/v1/domains/computer.03

响应示例

```
{ "catalog_version": "domains-2026-01", "item": { "id": "computer.03", "label": "人工智能与机器学习", "level": "category", ... }}
```

POST /v1/domains/{domain_id}/research

根据领域查询知乎专业问题或争议问题，供观点题生成使用。

项目	说明
是否需要登录	不需要登录（当前代码）
请求参数	路径参数 domain_id; count 默认8。

请求示例

```
/v1/domains/computer.03/research?count=8
```

响应示例

```
{ "domain_id": "computer.03", "items": [ { "question_id": "...", "title": "AI 会取代程序员吗?", "summary": "..."} ] }
```

前端提示：生产环境建议在路由层增加登录校验。

5. 性格测评

GET /v1/personality/questions

获取 Big Five/IPIP 50 题问卷。

项目	说明
是否需要登录	不需要登录（当前代码）
请求参数	无

响应示例

```
{ "instrument_version": "iipip-big-five-50-zh-v1", "model": "big_five", "scale": { "min": 1, "max": 5, "labels": [ "非常不符合", "较不符合", "一般", "较符合", "非常符合" ] }, "items": [ { "id": "E01", "text": "我是聚会中活跃的人。"} ] }
```

POST /v1/personality/assessments

提交性格测评答案并保存结果。

项目	说明
是否需要登录	通过 X-User-Id
请求参数	answers 为题目 ID 到1-5整数的映射；notes 可选；request_key 可选，用于幂等。

请求示例

```
{ "answers": { "E01": 4, "E02": 5, "A01": 3 }, "notes": "第一次测评", "request_key": "assessment-submit-1" }
```

响应示例

```
{ "assessment_id": "assessment_xxx", "status": "partial", "scores": { "extraversion": 7.5, "agreeableness": 5.0 }, "confidence": 0.15, "style_tags": [ "主动社交" ] }
```

前端提示：完整 50 题且无明显直线作答时状态为 completed；答案非法返回 422。

POST /v1/personality/skip

跳过性格测评，写入低置信度的 skipped 结果。

项目	说明
是否需要登录	通过 X-User-Id
请求参数	request_key 可选。

请求示例

```
{"request_key":"skip-1"}
```

响应示例

```
{"assessment_id":"assessment_xxx","status":"skipped","confidence":0.0,"scores":{"extraversion":null,"agreeableness":null,"conscientiousness":null,"neuroticism":null,"openness":null}}
```

GET /v1/personality/assessments/latest

读取当前用户最近一次性格测评。

项目	说明
是否需要登录	通过 X-User-Id
请求参数	无

请求示例

```
X-User-Id: user_123
```

响应示例

```
{"assessment_id":"assessment_xxx","status":"completed","scores":{"extraversion":6.8,...},"confidence":0.84}
```

6. 画像读取、修改与版本

GET /v1/twin/profile

读取当前用户的原型画像。

项目	说明
是否需要登录	通过 X-User-Id
请求参数	无

请求示例

```
X-User-Id: user_123
```

响应示例

```
{"revision":1,"status":"draft","data":{"facts":[],"domains":[],"stances":[],"style":{"},"memories":[],"personality":{"...},"evidence":[]}}
```

PATCH /v1/twin/profile

修改原型画像中的 data 字段。

项目	说明
是否需要登录	通过 X-User-Id
请求参数	body.data 为要合并的字段。

请求示例

```
{"data":{"facts":["后端工程师"],"style":{"tone":["直接"]}}}
```

响应示例

```
{"revision":2,"status":"draft","data":{"facts":["后端工程师"],"style":{"tone":["直接"]},...}}
```

前端提示：这是旧原型内存画像接口；正式 UUID 画像应通过 ProfileRepository 和版本机制写入。

POST /v1/twin/versions

发布当前原型画像为一个版本。

项目	说明
是否需要登录	通过 X-User-Id
请求参数	expected_revision，默认0。

请求示例

```
{"expected_revision":2}
```

响应示例

```
{"id":"version_xxx","user_id":"user_123","revision":2,"created_at":"...","card":{"spec":"chara_card_v2",...}}
```

前端提示：revision 不一致返回 409，前端应先重新 GET profile。

GET /v1/twin/versions/{vid}

读取指定画像版本。

项目	说明
是否需要登录	通过 X-User-Id
请求参数	路径参数 vid。

请求示例

```
/v1/twin/versions/version_xxx
```

响应示例

```
{"id":"version_xxx","user_id":"user_123","revision":2,"card":{"spec":"chara_card_v2",...}}
```

7. 数字分身初始化与心灵感应题

这些接口位于 runtime API，当前使用 X-User-Id

识别用户。初始化会话把性格、领域、观点题和社交题结果暂存在数据库，完成时调用 ProfileRepository 创建初始画像。

POST /v1/twin/initializations

创建或恢复当前用户唯一的初始化会话。

项目	说明
是否需要登录	通过 X-User-Id
请求参数	import_job_id 可选。

请求示例

```
{"import_job_id":"job_xxx"}
```

响应示例

```
{"id":"session_xxx","user_id":"user_123","status":"importing","current_step":"importing","input_data":{"generated_profile":{}}
```

前端提示：同一用户重复调用不会创建第二个会话。

GET /v1/twin/initializations/{session_id}

查询初始化进度和已保存输入。

项目	说明
是否需要登录	当前代码未在路由层校验用户归属，前端应只访问自己的 session_id
请求参数	路径参数 session_id。

请求示例

```
/v1/twin/initializations/session_xxx
```

响应示例

```
{"id":"session_xxx","status":"domain_pending","current_step":"domains","input_data":{"personality":{"...},"domains":{"...}}}
```

POST /v1/twin/initializations/{session_id}/{step}

保存 personality、domains、opinion-answers 或 social-answers 步骤。

项目	说明
是否需要登录	当前代码未在路由层校验用户归属
请求参数	路径参数 session_id、step；body.data 为步骤数据。

请求示例

```
{"data":{"answers":{"E01":4},"notes":"..."}}
```

响应示例

```
{"id":"session_xxx","status":"personality_pending","current_step":"personality","input_data":{"personality":{"answers":{"E01":4}}}}
```

前端提示：前端应按初始化流程顺序提交，非法 session 返回 404。

POST /v1/twin/initializations/{session_id}/complete

用基础身份和已保存结果创建初始画像。

项目	说明
是否需要登录	当前代码未在路由层校验用户归属
请求参数	body.identity 可包含 display_name、summary、occupation、location、age、privacy_level、extra。

请求示例

```
{"identity":{"display_name":"小明","summary":"后端工程师","occupation":"软件工程师","privacy_level":"private"}}
```

响应示例

```
{"avatar_id":"...","version_id":"...","version_no":1,"status":"initialized"}
```

前端提示：当前生成器主要复用测评和用户输入；知乎资料、LLM 提炼的完整闭环仍需后续接入。

POST /v1/twin/mind-reading/{dialogue_run_id}

保存一条对话后待用户选择的心灵感应题。

项目	说明
是否需要登录	通过 X-User-Id
请求参数	body.avatar_id 必须提供；其余题干、选项和 agent_option_id 放在 body。

请求示例

```
{ "avatar_id": "avatar_b", "question": "遇到反例时会怎么做？", "options": [{"id": "a", "text": "修正观点"}, {"id": "b", "text": "坚持原观点"}], "agent_option_id": "a" }
```

响应示例

```
{ "id": "feedback_xxx", "question": {...}, "status": "pending" }
```

POST /v1/twin/mind-reading/{question_id}/answer

提交心灵感应题选择。

项目	说明
是否需要登录	当前代码未在路由层校验用户归属
请求参数	body.option_id。

请求示例

```
{ "option_id": "a" }
```

响应示例

```
{ "id": "feedback_xxx", "is_match": true, "status": "answered" }
```

8. Agent 聊天组查询

聊天组是一次双 Agent 对话函数调用的索引；完整正文仍来自 chat_messages。用户只要是发起方或被邀请方，就能看到这一组。

GET /v1/twin/chat-groups/count

统计当前用户参与的聊天组数量。

项目	说明
是否需要登录	通过 X-User-Id
请求参数	status 可选，例如 completed/running/failed。

请求示例

```
?status=completed
```

响应示例

```
{ "count": 12 }
```

GET /v1/twin/chat-groups

分页列出当前用户参与的聊天组和双方基础信息。

项目	说明
是否需要登录	通过 X-User-Id
请求参数	page 从1开始；page_size 1-100，默认20；status 可选。

请求示例

?page=1&page_size=20&status=completed

响应示例

```
{
  "items": [
    {
      "chat_no": "chat_xxx",
      "dialogue_run_id": "run_xxx",
      "conversation_id": "conv_xxx",
      "initiator": {
        "user_id": "u1",
        "avatar_id": "a1",
        "display_name": "Agent A"
      },
      "invited": {
        "user_id": "u2",
        "avatar_id": "a2",
        "display_name": "Agent B"
      },
      "status": "completed",
      "started_at": "...",
      "ended_at": "...",
      "message_count": 12
    }
  ],
  "page": 1,
  "page_size": 20,
  "total": 35
}
```

GET /v1/twin/chat-groups/{chat_no}/messages

按顺序读取一组聊天的完整消息。

项目	说明
是否需要登录	通过 X-User-Id，必须是发起方或被邀请方
请求参数	路径参数 chat_no。软删除消息不会返回。

请求示例

/v1/twin/chat-groups/chat_abc/messages

响应示例

```
{
  "chat_no": "chat_abc",
  "dialogue_run_id": "run_xxx",
  "conversation_id": "conv_xxx",
  "status": "completed",
  "initiator": {
    "user_id": "u1",
    "avatar_id": "a1",
    "display_name": "Agent A"
  },
  "invited": {
    "user_id": "u2",
    "avatar_id": "a2",
    "display_name": "Agent B"
  },
  "messages": [
    {
      "message_id": "m1",
      "sequence_no": 1,
      "sender_avatar_id": "a1",
      "sender_name": "Agent A",
      "content": "你好",
      "message_type": "text",
      "sent_at": "...",
      "metadata": {}
    }
  ]
}
```

前端提示：非参与用户和不存在的 chat_no 都返回 404，避免泄露记录是否存在。

9. 供业务代码调用、但不是 HTTP 接口的函数

下面函数是后端内部能力，前端不能直接访问。它们由服务端编排流程调用，也不会自动暴露给 Agent。

函数	作用	前端是否直接调用
build_context(user_id, question, ...)	固定画像直查 PostgreSQL，长内容用 PostgreSQL 关键词 + Chroma 混合检索，组装结构化上下文。	否
run_agent_dialogue(...)	运行一次双 Agent 交替对话，写入 chat_messages、agent_dialogue_runs 和 agent_chat_groups。	否；当前需由业务服务触发
create_behavior_memory_tool(...)	绑定 user_id/avatar_id 的行为记忆 LangGraph 工具。	否
InitializationService.complete(...)	使用已保存初始化数据创建初始画像。	通过 HTTP 初始化接口间接调用
ChatGroupService.*	查询聊天组数量、列表和完整消息。	通过 HTTP 聊天组接口间接调用

10. 常见错误与前端处理

- 401：未登录、会话无效或知乎授权过期。引导用户重新授权。
- 404：资源不存在或用户无权访问。聊天详情接口故意统一返回 404，前端不要区分“不存在”和“无权限”。
- 409：revision 冲突。重新读取最新 profile 或领域选择后再提交。
- 422：请求参数格式不合法，例如题目答案不在1-5、领域 ID 不可选。
- 429/5xx：知乎频率限制、网络错误或服务暂时不可用；只有 error.retryable=true 时才自动重试。

11. 当前实现边界

- 当前仓库已经提供聊天组查询接口，但没有单独的 HTTP POST /dialogues 路由；启动双 Agent 对话仍需由业务代码调用 run_agent_dialogue。
- runtime 初始化接口的部分路由目前缺少用户归属校验，前端必须保存并只使用当前用户自己的 session_id。
- 正式 OAuth 接口使用 HttpOnly Cookie；/v1/* 原型接口依赖 X-User-Id，二者不要混用为同一种生产认证方案。
- 接口返回结构以当前代码为准；如果后端后续统一响应包装，前端应以版本号或 OpenAPI 文档更新为准。